Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

COURSE / PROJECT REVIEW-1 (2026-27)

Supervisor: Assistant Professor Shaik Asif, Department of CSE	**Team No.: 17**
Project Title: Dockerized AI Stack using docker-compose and Hardened Images	

TEAM MEMBERS TABLE

Roll Number	Student Name	Signature
2520030599	D. Hemasai	
2520030560	M. Nikhil Reddy	

1. Abstract

This project proposes a Dockerized AI Stack using Docker Compose and Hardened Images, an infrastructure-level DevSecOps architecture that deploys Artificial Intelligence inference services and vector databases in isolated, production-hardened containers. Deploying AI models directly onto host systems often causes dependency conflicts and security exposure, while default Docker containers run as root with bloated OS packages. In our architecture, Docker Compose orchestrates three decoupled microservices—a Reverse Proxy Gateway, an AI Inference Worker, and a Vector Database—inside a private internal bridge network (internal-net) where only the gateway exposes a public port (Port 8080). Grounded in NIST SP 800-190 and CIS Docker Benchmark standards, every container enforces minimal multi-stage base images, unprivileged non-root execution (UID 10001), complete Linux capability dropping (cap_drop: ALL), read-only root filesystems paired with memory-backed tmpfs directories (/tmp) for AI caches, and in-memory Docker Secrets (/run/secrets). At this Review-

1 stage, topic selection, a literature survey of foundational papers (Combe et al., 2016; Souppaya et al., NIST SP 800-190; Shu et al., 2017), and the 3-tier system architecture design are complete.

---

2. Problem Statement

Deploying AI applications on host machines leads to severe Python and CUDA library conflicts across environments. While standard Docker containers solve portability, default community images are bloated (~1,012 MB), carry hundreds of unpatched OS vulnerabilities, and execute as the root user (UID 0), exposing the host to container-breakout and privilege-escalation attacks. Existing studies on container security, such as Combe et al. (2016) and NIST SP 800-190 (2017), define general threat models and hardening baselines for stateless web microservices but do not address AI-specific runtime failures—such as framework cache crashes under read-only filesystems or volume permission errors for non-root vector databases. This is the research gap identified from our literature survey. The project aims to build a reproducible, single-command AI stack that enforces strict container hardening without breaking stateful AI inference.

---

3. Objectives

1. To orchestrate a 3-tier AI stack (Reverse Proxy, AI Inference Worker, and Vector Database) using a single Docker Compose blueprint.

2. To minimize container image footprints and CVE attack surfaces using multi-stage minimal hardened base images.

3. To enforce least-privilege runtime security using non-root execution (UID 10001), cap_drop: ALL, and no-new-privileges.

4. To achieve root filesystem immutability (read_only: true) while supporting AI runtime caches via volatile RAM mounts (tmpfs).

5. To isolate backend AI services inside a private bridge network (internal-net) and protect credentials using Docker Secrets.

6. To benchmark the hardened stack against a standard Python deployment across image size, CVE count, and startup latency.

---

4. Proposed Methodology

The system is designed in five structured stages: (1) Multi-stage Dockerfiles compile dependencies in a builder stage and copy only required runtime artifacts into a minimal hardened base image owned by unprivileged user UID 10001. (2) Docker Compose V2 defines three isolated services connected over a private internal bridge network (internal-net). (3) Only the Reverse Proxy Gateway exposes Port 8080 to external clients, routing requests via internal Docker DNS to the AI Inference Worker (port 5000) and Vector Database (port 6333). (4) Runtime hardening locks are applied in docker-compose.yml: read_only: true root filesystems, RAM-backed tmpfs mounts at /tmp (with HF_HOME=/tmp/cache), cap_drop: [ALL], and read-only model volume mounts (:ro). (5) Sensitive API keys are mounted securely in volatile memory via /run/secrets instead of plaintext .env files, and the stack is verified using Trivy vulnerability scanning and startup benchmarks.

---

5. Core Concepts / Hardening Techniques Used

Concept / Technique	Purpose in the Project
Multi-Stage Minimal Base Images	Strip compilers, shells, and package managers to shrink image size and eliminate CVEs.
Docker Compose V2 Orchestration	Manage multi-container networking, volumes, healthchecks, and deployment via one YAML file.
Private Bridge Network (internal-net)	Isolate AI Inference Worker and Vector Database with zero public host port bindings.
Non-Root User (UID 10001) & cap_drop: ALL	Enforce least-privilege execution and prevent container-breakout or root escalation.
Immutable Filesystem (read_only: true) + tmpfs	Prevent runtime tampering while allowing fast in-memory AI model caching at /tmp.
Docker Secrets (/run/secrets)	Inject API keys securely in memory without exposing plaintext environment variables.

---

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030599	D. Hemasai	1) Project title clarity, abstract, reference papers, and literature survey. 2) Multi-stage Dockerfiles, minimal hardened base images, and non-root (UID 10001) configuration.
2520030560	M. Nikhil Reddy	1) Societal needs, novelty, and project feasibility analysis. 2) Docker Compose orchestration, internal-net isolation, tmpfs/secrets setup, and Trivy benchmark verification.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Docker Engine & Docker Compose V2	Used to build hardened images and orchestrate the 3-tier containerized AI stack.
Minimal Hardened Linux Images (Alpine / Distroless)	Used to minimize OS package bloat and eliminate critical CVEs.
FastAPI & PyTorch / Hugging Face	Used to serve the containerized AI Inference Worker.
Vector Database (Qdrant)	Used to store and query high-dimensional vector embeddings inside internal-net.
Reverse Proxy Gateway (Nginx / Caddy)	Used as the sole public entry point (Port 8080) routing traffic to internal services.
Trivy & GitHub	Used for static CVE vulnerability scanning, version control, and Review-1 documentation.

8. Expected Outcome

The project is expected to deliver a fully reproducible, single-command (docker compose up -d) hardened AI deployment stack that eliminates dependency conflicts and protects sensitive AI workloads from container-level attacks. Compared to a standard Python 3.11 deployment,

the hardened architecture is designed to reduce image size by over 80%, eliminate all critical CVEs, and improve cold startup speed while maintaining full AI inference functionality under read-only, non-root constraints.